Yes. **CMS (Concurrent Mark-Sweep)** is an important missing piece because it explains *why G1 was introduced* and why modern collectors moved toward concurrent compaction.

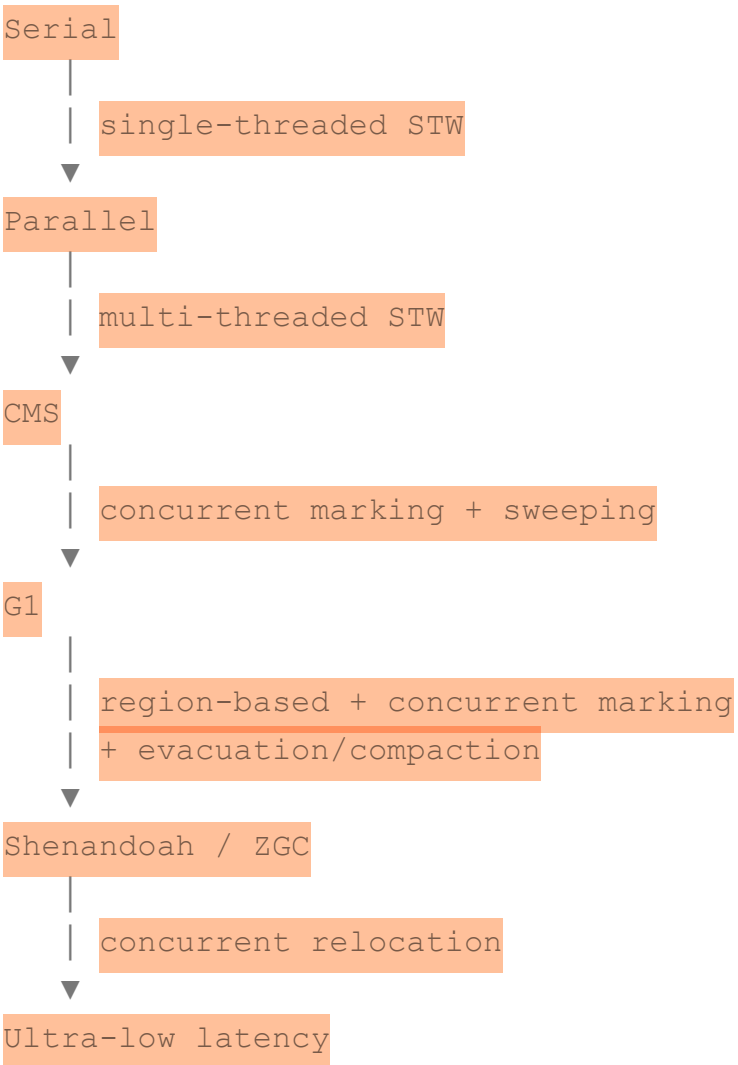
> **CMS tries to minimize STW pauses by doing most of the marking and sweeping concurrently with the application, but it does not compact the old generation.**

CMS was deprecated in JDK 9 and removed in **JDK 14**, so it is primarily a historical/interview topic today.

---

1. Where CMS fits

The evolution becomes:



CMS's major innovation was:

> **"Let the application continue while GC performs most of its work."**

---

2. What does CMS stand for?

CMS = Concurrent Mark-Sweep

There are three important words:

Concurrent

GC runs concurrently with application threads.

Mark

Identify reachable/live objects.

Sweep

Reclaim unreachable objects.

The important omission is:

> **No compact.**

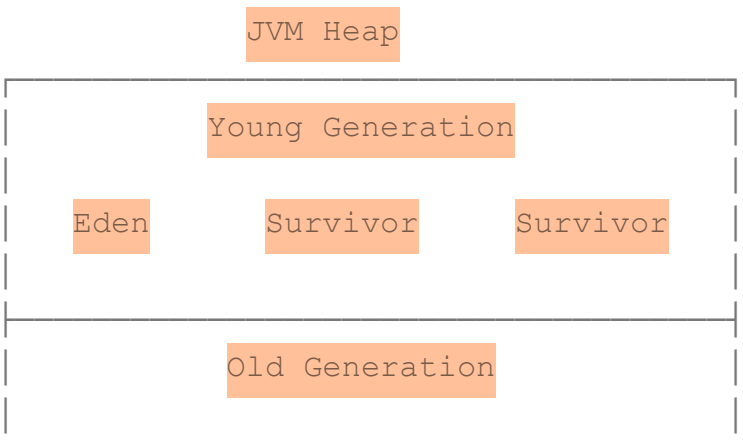
That's CMS's major weakness.

---

3. Heap Structure

CMS is generational.

Conceptually:





CMS primarily deals with the **Old Generation**.

Young collections are handled separately, traditionally with a copying collector.

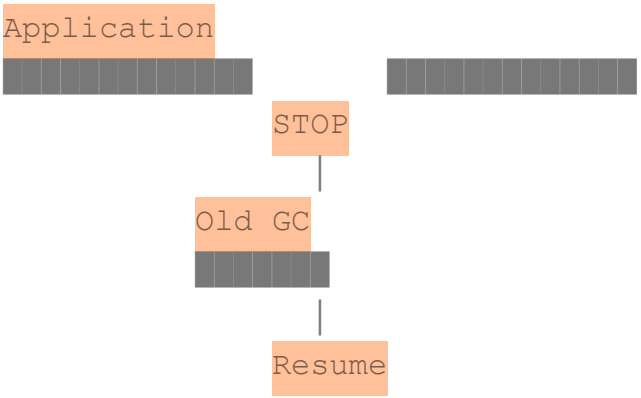
---

4. Why Did CMS Exist?

Suppose an application has:

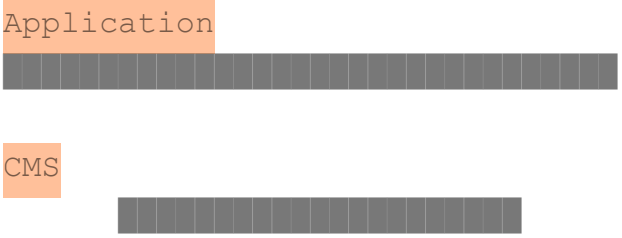
```
Large heap
+
Long-lived objects
+
Strict latency requirements
```

With Parallel GC:



If Old Gen is large, that pause could be substantial.

CMS instead does:



Most of the work happens while the application continues.

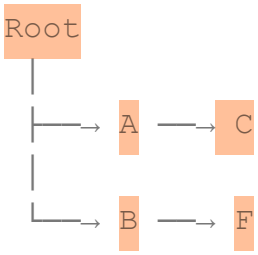
---

5. Let's Build an Example

Suppose Old Gen contains:



Object graph:



Therefore:

```
Live:
A B C F
```

```
Garbage:
D E G H
```

CMS needs to identify and reclaim:

```
D E G H
```

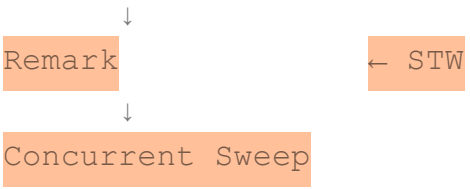
without stopping the application for the entire operation.

---

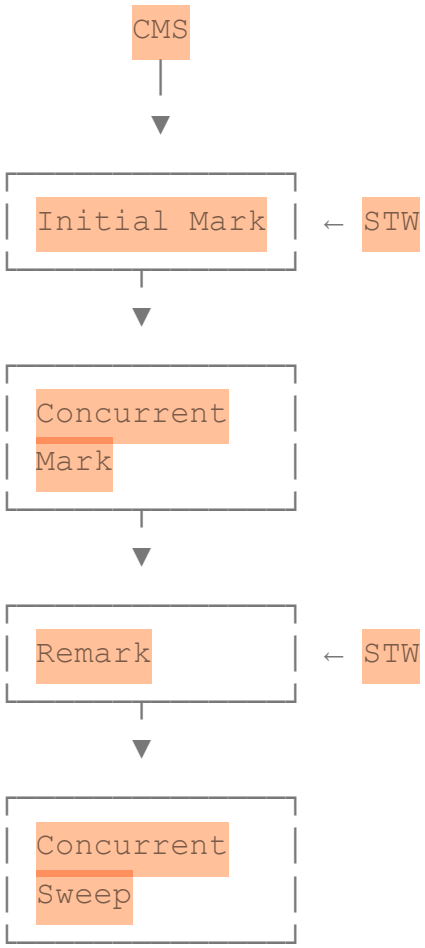
6. CMS Has Four Major Phases

The simplified CMS lifecycle is:

```
Initial Mark        ← STW
      ↓
Concurrent Mark
```



This is the diagram you should memorize.

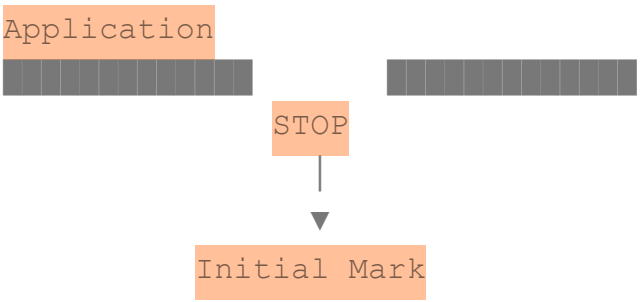


Let's go through it.

---

7. Phase 1 – Initial Mark

Application threads briefly stop.



```
        |
     Resume
```

CMS identifies objects directly reachable from GC roots.

For example:

```
Root
 |
 ├──→ A
 |
 └──→ B
```

CMS marks:

```
A ✓
B ✓
```

The key point:

> **Initial Mark is STW, but intentionally short.**

---

8. Phase 2 — Concurrent Mark

Now the application resumes.

CMS traverses the object graph concurrently.

```
Application
████████████████████████████

CMS
        ████████████████████
```

It follows:

```
A → C

B → F
```

and marks:

```
A ✓
B ✓
C ✓
F ✓
```

while D/E/G/H remain unmarked.

---

9. But What If the Application Changes References?

This is where CMS becomes interesting.

Suppose CMS has already scanned:

```
A → B
```

and marked B.

Then application changes:

```
A → C
```

while concurrent marking is happening.

Or consider the opposite problem:

```
A initially does NOT point to C
```

GC hasn't marked C yet.

Then application does:

```
A → C
```

If A is already black/fully processed, we could have:

```
Black A → White C
```

which could cause C to be missed.

CMS therefore uses **write barriers/card marking** and its remark phase to account for mutations that occurred during concurrent marking.

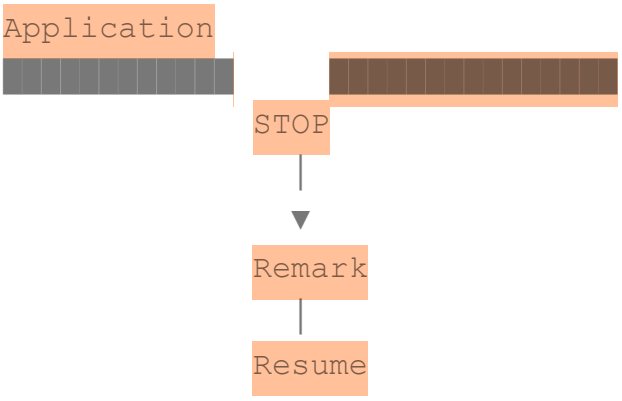
This connects directly to the **tri-color invariant** we discussed earlier.

---

10. Phase 3 — Remark

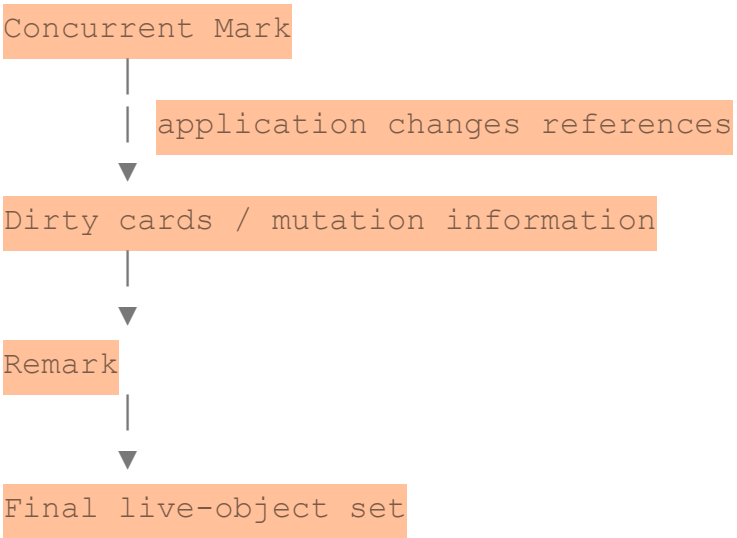
Eventually CMS needs to finalize marking.

Application threads briefly stop:



The remark phase processes changes that occurred while concurrent marking was running.

Conceptually:



This is why CMS can remain mostly concurrent without losing objects.

---

11. Phase 4 — Concurrent Sweep

Now CMS knows which objects are garbage.

For our example:

```
Live:
A B C F

Garbage:
D E G H
```

CMS sweeps concurrently:

```
Application
████████████████████████

CMS
            ████████████████
```

It reclaims:

```
D
E
G
H
```

---

12. The Big Problem: No Compaction

After sweep:

```
┌─────────────────────────────┐
│ A │ B │ C │   │   │ F │   │   │
└─────────────────────────────┘
```

```
        ↑          ↑
      free       free
```

The memory is fragmented.

Suppose we now want to allocate a large object:

```
Object = 20 MB
```

Even if the heap has:

```
30 MB total free
```

it might not have:

```
20 MB contiguous free
```

So CMS can have a **fragmentation problem**.

---

13. Why Can't CMS Just Compact?

Because compaction means moving objects:

```
A @ 1000 → A @ 5000
B @ 2000 → B @ 6000
```

and then updating all references to them.

Doing that concurrently is considerably more complicated.

CMS intentionally chose:

> **Avoid concurrent compaction to keep pause times low.**

The consequence is:

> **Fragmentation can eventually force a Full GC.**

---

14. The CMS Full GC Problem

Imagine:

```
Old Gen
```



Free memory:

```
Lots of small holes
```

but no sufficiently large contiguous block.

CMS may fail to satisfy a large allocation.

The JVM may then fall back to a **stop-the-world Full GC**, typically using a compacting collector.

Conceptually:

```
CMS
 │
 ├── Concurrent Mark
 ├── Remark
 ├── Concurrent Sweep
 │
 │  fragmentation
 │
 ▼
Allocation failure
 │
 ▼
Full GC
 │
 ▼
STW Compaction
```

This is one of the biggest reasons CMS was eventually superseded.

---

15. Another Problem: Floating Garbage

There's another important CMS concept: **floating garbage**.

Suppose:

```
A → B
```

CMS determines B is live.

Then while CMS is marking:

```
A removes → B
```

Now B is actually unreachable.

But CMS may already have marked it.

Therefore:

```
B = garbage
```

but CMS doesn't necessarily reclaim it during this collection.

It becomes **floating garbage** and can be reclaimed during a later cycle.

This is generally acceptable because:

> **Incorrectly keeping garbage for one cycle is safe; incorrectly reclaiming a live object is not.**

This is a very important GC principle.

---

16. Complete CMS Example

Let's walk through a complete cycle.

Initial heap:

```
Old Generation
```

```
A B C D E F G H
```

Graph:

```
Root
 │
 ├── A ──→ C
 │
 └── B ──→ F
```

So:

```
Live:     A B C F
Garbage: D E G H
```

Phase 1 — Initial Mark

```
STW
 ↓
Mark A and B
 ↓
Resume
```

Phase 2 — Concurrent Mark

```
Application ████████████████████
CMS           ████████████████
```

CMS discovers:

```
A → C
B → F
```

and marks:

```
A B C F
```

Application mutation

While CMS runs:

```
B → F
```

becomes:

```
B → G
```

CMS's barriers record relevant mutation information.

Phase 3 — Remark

```
STW
 ↓
Process mutation information
 ↓
Finalize marking
 ↓
Resume
```

Phase 4 — Concurrent Sweep

CMS reclaims:

```
D E
```

and other garbage.

The heap might become:

```
┌─────────────────────────────────┐
| A | B | C |   |   | F |   |   |
```



Notice:

```
A B C F
```

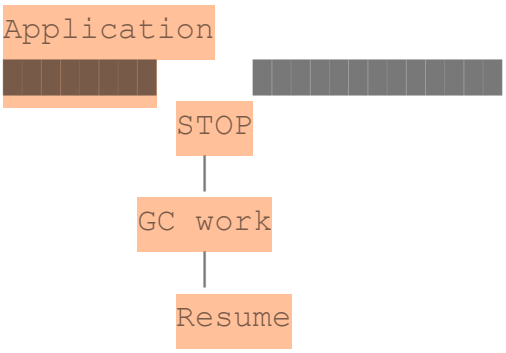
remain, but they're not necessarily packed together.

---

17. CMS vs Parallel GC

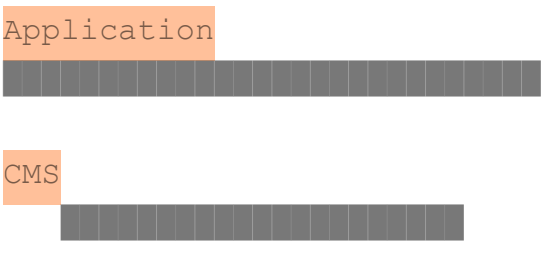
This is a critical interview comparison.

Parallel GC



Most GC work occurs during STW.

CMS



Only:

```
Initial Mark
Remark
```

are STW.

The expensive:

```
Concurrent Mark
Concurrent Sweep
```

happen alongside the application.

---

18. CMS vs G1

Now we can see why G1 was such an important evolution.

	CMS	G1
Generational	Yes	Yes
Region-based	No	**Yes**
Concurrent marking	Yes	Yes
Concurrent sweep	Yes	Different model
Compaction	❌ No	**Yes, via evacuation**
Fragmentation	**Problem**	Much better controlled
Remembered sets	Card-table based	**Per-region remembered sets**
Mixed collection	No	**Yes**
Pause target	Less predictable	**Pause-time oriented**
Modern status	Removed	Modern/default choice in many JVMs

The key evolution:

```
CMS:

Concurrent Mark
      ↓
Concurrent Sweep
      ↓
```

```
Fragmentation ❌
```

```
G1:

Concurrent Mark
      ↓
Select regions
      ↓
Evacuate live objects
      ↓
Compact/reclaim regions
```

---

19. Why G1 Replaced CMS

CMS had several problems:

① Fragmentation

```
FREE  LIVE  FREE  LIVE  FREE
```

Eventually problematic.

② Full GC fallback

Fragmentation could force:

```
STW Full GC
```

which defeats the low-latency objective.

③ Complex tuning

CMS required tuning around:

- Concurrent cycle initiation
- Old-gen occupancy
- Number of concurrent threads

- Promotion behavior
- Fragmentation
- Allocation rates

④ CPU competition

CMS runs concurrently:

```
Application → CPU
CMS         → CPU
```

which can reduce application throughput.

⑤ Floating garbage

Some garbage survives one CMS cycle.

---

20. Why G1 Is Better Conceptually

CMS says:

> "I'll avoid moving objects so I can stay concurrent."

G1 says:

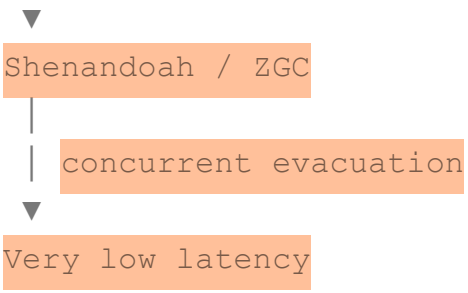
> "I'll divide the heap into regions and move objects from selected regions during controlled pauses."

Modern collectors go further:

> "I'll move objects **concurrently**."

That gives us:

```
CMS
 |
 | no compaction
 ▼
G1
 |
 | STW evacuation
```



---

21. CMS and Tri-Color Marking

This connects directly to your first question.

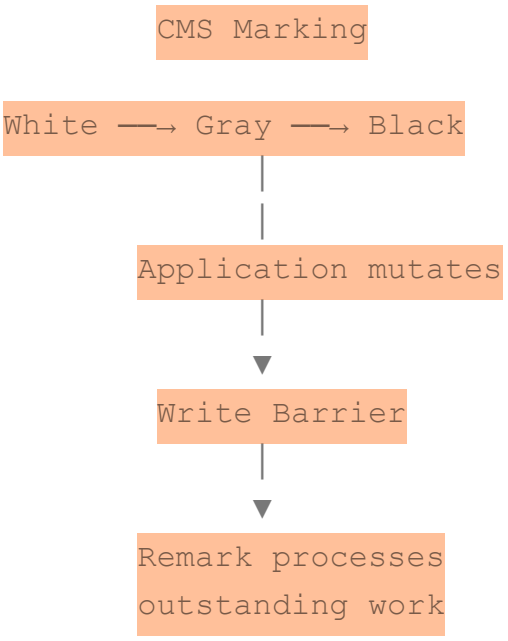
During concurrent marking:

```
White = not yet discovered
Gray  = discovered, not fully scanned
Black = fully scanned
```

CMS is vulnerable to mutations because application threads continue running.

Therefore it uses barriers to maintain marking correctness.

Conceptually:



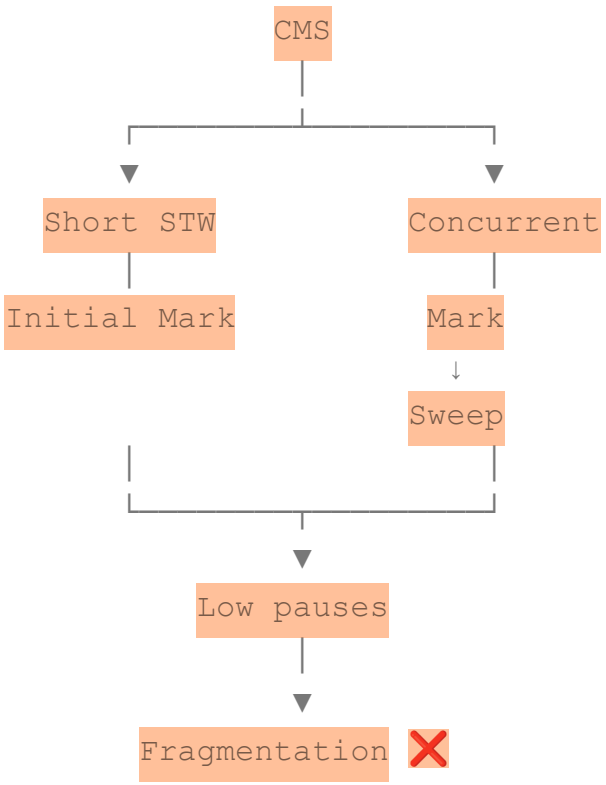
So the concepts you've been learning are directly connected:

```
Tri-color marking
        ↓
Concurrent marking
        ↓
Write barriers
        ↓
Remark
        ↓
Concurrent sweep
```

---

22. The Most Important CMS Mental Model

Think:



The **one sentence to remember** is:

> **CMS minimizes pause times by performing marking and sweeping concurrently, but because it doesn't compact the old generation, fragmentation and eventual Full GC are its fundamental weaknesses.**

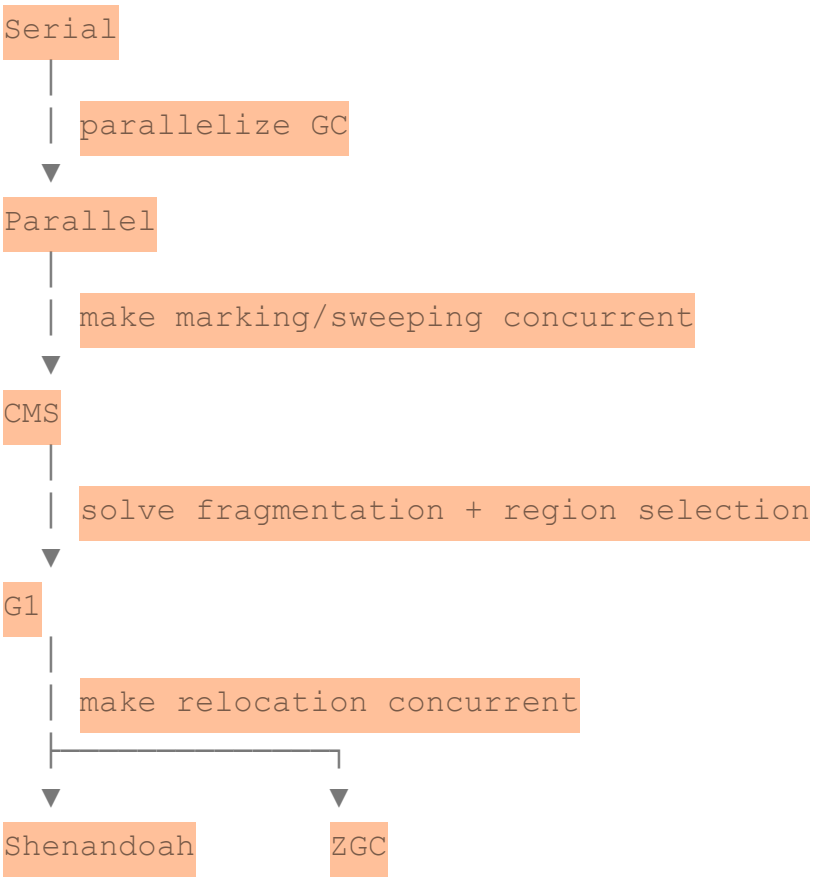
---

23. Your GC Map Is Now Complete

You can now connect all five collectors:

Collector	Main Technique	Main Problem
Serial	STW + single thread	Long pauses
Parallel	STW + many threads	Still long pauses
CMS	Concurrent mark/sweep	Fragmentation
G1	Regions + evacuation	Some STW evacuation
Shenandoah	Concurrent evacuation	Barrier/CPU overhead
ZGC	Colored ptr + barriers	Barrier/CPU complexity

And the **evolutionary story** is particularly useful in a Staff/Architect interview:



That story is more valuable in an interview than memorizing isolated GC names because it lets you explain **why each collector exists and what problem the next generation of collectors was designed to solve.**